

Лабораторная работа №6

Тема: Оптимизация памяти, типы данных Categorical, Scikit-learn Pipelines и быстрые форматы хранения

Данные: `olist_orders_dataset.csv`, `olist_customers_dataset.csv`, `olist_order_items_dataset.csv`

Контекст задачи

Задача любого хорошего маркетплейса — обрабатывать миллионы заказов так, чтобы пользователи оставались довольны. Стандартные методы Pandas могут потреблять слишком много RAM и работать медленно. Вам нужно оптимизировать хранение данных и создать воспроизводимый *Pipeline*, пригодный для обучения моделей машинного обучения на этих данных.

Задание 1: Оптимизация памяти (Categorical Data)

Текстовые данные о штатах и статусах заказов занимают в памяти значительно больше места, чем могли бы.

1. **Измерение:** Оцените текущее потребление памяти вашим объединенным датасетом с помощью команды `df.info(memory_usage='deep')`.
2. **Трансформация:** Переведите колонки `customer_state`, `order_status` и `product_category_name` в тип `category`.
3. **Сравнение:** Замерьте объем памяти «до» и «после». *Ожидаемый результат — экономия от 50% до 80% RAM.*

Задание 2: Автоматизация подготовки (Sklearn Pipelines)

Чтобы избежать утечки данных (*data leakage*) и упростить деплой, нужно собрать предобработку в один объект.

Создайте `ColumnTransformer`, который включает:

- Для численных признаков (`price`, `freight_value`): заполнение пропусков медианой и масштабирование (`StandardScaler`).
- Для категориальных признаков: *One-Hot Encoding* или *Ordinal Encoding*.

Оберните это в `Pipeline` и протестируйте его на небольшом фрагменте данных.

Задание 3: Высокопроизводительные запросы (`pd.query`)

В сложных системах читаемость кода и скорость фильтрации имеют решающее значение. Реализуйте сложный фильтр (например: «заказы из штатов SP или RJ со стоимостью выше 150 BRL, сделанные в 2018 году») двумя способами:

1. Через классические булевы маски.
2. Через метод `.query()`.

Используйте `%timeit`, чтобы проверить, какой метод работает быстрее на вашем объеме данных.

Задание 4: Переход на бинарные форматы (HDF5 / Parquet)

CSV — это текстовый формат, который теряет информацию о типах данных (ваши `datetime` и `category` снова станут строками при чтении).

1. Сохраните финальный датасет в формате **HDF5** или **Parquet**.
2. Продемонстрируйте разницу: сравните размер файла на диске и скорость загрузки обратно в Pandas по сравнению с исходным CSV.

Требования к сдаче

- [] **Memory Efficient:** В тетрадке зафиксировано значительное снижение потребления памяти после типизации.
- [] **Reusable Code:** Пайплайн успешно преобразует «сырые» данные в матрицу признаков, готовую для обучения модели (например, **XGBoost**).
- [] **Storage Logic:** Обоснован выбор бинарного формата для хранения (преимущества *Parquet* или *HDF5* для данного проекта).